

# Graph-Based Contract Sensing Framework for Smart Contract Vulnerability Detection

Yan Pang<sup>ID</sup>, Xiangfu Liu, Teng Huang<sup>ID</sup>, Yile Hong<sup>ID</sup>, Jiahui Huang<sup>ID</sup>, Sisi Duan<sup>ID</sup>, and Changyu Dong<sup>ID</sup>

**Abstract**—Smart contract vulnerabilities have led to significant economic losses, threatening blockchain security and development. Graph neural network (GNN)-based approaches, which capture the structural properties of contracts and leverage code dependencies to better understand contract behavior, have become widely used for vulnerability detection. However, these approaches face challenges in losing valuable information during graph construction and failing to capture rich semantic content, while traditional GNNs struggle with long-range dependencies and global context in complex contract graphs. To address these challenges, we propose ConSense, a GNN-based Contract Sensing Framework for Smart Contract Vulnerability Detection. ConSense comprises two core components: the smart contract graph generator, which constructs contract graphs while retaining both structural and semantic information, and ExploreFormer, which effectively integrates local and global context using advanced attention mechanisms for vulnerability detection. Comprehensive experimental evaluations were performed on the IR-ESCD and SCVHunter-SCD datasets. For instance, the IR-ESCD benchmark—which encompasses eight distinct vulnerability categories—demonstrates that ConSense attains an average detection accuracy of 97.74%, with a mean processing time of 0.648 seconds per contract. These results signify a statistically significant improvement over state-of-the-art methods in both precision and computational efficiency.

**Index Terms**—Smart contract, vulnerability detection, graph learning.

## I. INTRODUCTION

THE rapid advancement of general-purpose blockchain technology, epitomized by smart contracts, has enabled complex business logic to be executed on-chain, extending the capabilities of blockchain beyond Bitcoin's simple transaction scripts [1]. With applications in finance [2], gaming [3], and the Internet of Things (IoT) [4], smart contracts have become

integral to numerous industries. However, the soaring market value of cryptocurrencies has also attracted cybercriminals, leading to high-profile attacks such as the DAO incident [5], where a reentrancy vulnerability was exploited to steal over \$70 million in Ether, and the \$100 million loss suffered by Binance in 2022 due to vulnerabilities in its cross-chain bridge [6]. These incidents underscore the critical need for robust vulnerability detection mechanisms in smart contracts to safeguard the security of blockchain transactions and prevent further financial losses.

The detection of smart contract vulnerabilities in blockchains has traditionally relied on techniques such as symbolic execution [7], [8], [9], formal verification [10], [11], [12], static analysis [13], [14], [15] and fuzz testing [16], [17], [18], which utilize predefined, vulnerability-specific rules to identify security threats. While effective to some extent, these methods suffer from high spatial complexity and extended detection times, making them insufficient to address the growing demand for rapid vulnerability detection in real-world blockchain applications. To overcome these limitations, researchers have increasingly turned to deep learning approaches, which offer superior feature extraction capabilities and can automatically identify vulnerabilities. Notable examples include SafeSC [19], which employs Long Short-Term Memory (LSTM [20]) for opcode-focused vulnerability detection, and ReChecker [21], which leverages a Bidirectional LSTM with an attention mechanism to detect reentrancy vulnerabilities specifically. These deep learning models have demonstrated impressive performance improvements by learning latent features from contract data. However, their reliance on the textual representation of smart contracts overlooks the unique structural characteristics of code, fails to utilize the rich control flow and data flow semantics, which is particularly detrimental for detecting vulnerabilities that depend on the global structure of the code and the complex relationships between program elements, such as integer overflow and reentrancy, highlighting an important area for further research in vulnerability detection methodologies.

Recent approaches [22], [23], [24], [25] have sought to better leverage the structural information in smart contracts by converting contract code into graph structures and applying GNNs for vulnerability detection. For example, TMP [22] transforms contract function source code into contract graphs, represents program elements as nodes, highlights critical function invocations or variables through graph normalization, and uses a temporal message propagation GNN for vulnerability detection. Building on TMP, AME [23] combines contract graph information with expert knowledge in an interpretable manner to

Received 13 March 2025; revised 22 June 2025; accepted 17 July 2025. Date of publication 30 July 2025; date of current version 13 November 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFB2704300, in part by the Guangdong Natural Science Foundation under Grant 2025A1515010276, and in part by the Science and Technology Projects in Guangzhou 2024 Guangzhou School (Institute) Enterprise Joint Funding Project under Grant 2024A03J0166. Recommended for acceptance by E. Ferrari. (Corresponding authors: Teng Huang; Changyu Dong.)

Yan Pang, Xiangfu Liu, Teng Huang, Yile Hong, Jiahui Huang, and Changyu Dong are with the School of Artificial Intelligence, Guangzhou University, Guangzhou 511370, China (e-mail: yanpang@gzhu.edu.cn; xiangfu.liu@gzhu.edu.cn; huangteng1220@gzhu.edu.cn; yile.hong@gzhu.edu.cn; jiahuihuang@gzhu.edu.cn; changyu.dong@gzhu.edu.cn).

Sisi Duan is with the Institute for Advanced Study, Tsinghua University, Beijing 100190, China (e-mail: duansisi@tsinghua.edu.cn).

Digital Object Identifier 10.1109/TBDDATA.2025.3594303

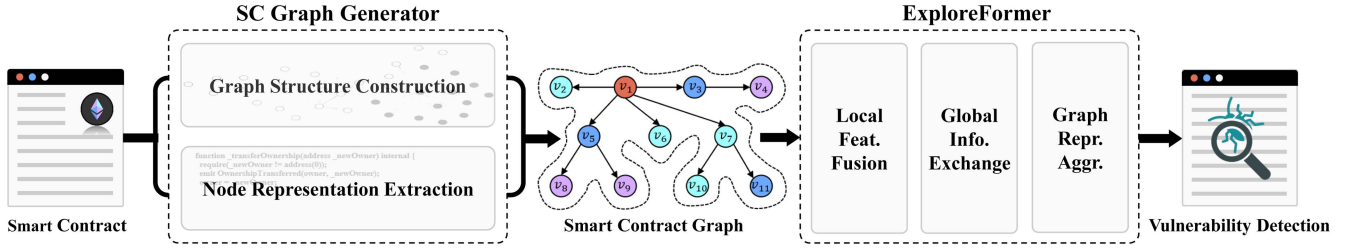


Fig. 1. The Consense consists of two core components: the SC Graph Generator (zoomed in Fig. 2), which constructs contract graphs by preserving both structural and semantic information from function call relationships, and ExploreFormer (zoomed in Fig. 3), a structure-aware GNN that processes these graphs by combining local node information with global context through attention mechanisms for vulnerability detection. "Feat.": Feature; "Info.": Information; "Repr.": Representation; "Aggr.": Aggregation.

enhance detection accuracy. Despite their advancements, these approaches encounter two major limitations: (1) During the contract graph construction phase, auxiliary nodes are removed to handle multiple node types, with their features aggregated into primary nodes. While this strategy focuses the graph on critical variables and functions, it risks losing valuable information. Additionally, these methods define contract graph nodes solely based on dependencies and node types, ignoring the corresponding source code snippets and the rich semantic information embedded within contract statements. (2) The use of traditional GNNs to analyze contract graphs restricts these methods to local information propagation, which is constrained by the receptive fields of GNNs when processing complex graph structures [26]. As a result, node representations may fail to capture information from more distant nodes, making it challenging to effectively aggregate global contextual information, especially in contract graphs with intricate function call relationships.

We propose ConSense, a Contract Sensing Framework for Smart Contract Vulnerability Detection, designed to address key limitations in current methods, particularly the loss of information during graph construction and the failure to capture global contextual relationships in contract graphs fully. As illustrated in Fig. 1, the ConSense is built around two core components: the smart contract (SC) graph generator and ExploreFormer. The SC Graph generator models the function call relationships within contracts, ensuring that all contract elements are retained while preserving the global structure. Unlike previous methods, which remove auxiliary nodes and risk losing valuable information, ConSense preserves all nodes and their relationships. To address the issue of overlooking intrinsic semantic information in contract statements, ConSense performs an in-depth analysis of our refined 91,119 smart contract source codes, using pretrained large-scale language models (LLMs) to extract the semantic information embedded in each contract element, enriching the graph node representations with both structure and semantics.

The second core component, ExploreFormer, solves the issue of traditional GNNs being limited by local information propagation. ExploreFormer is a structure-aware graph learning module designed to integrate both local and global contextual relationships through advanced attention mechanisms on generated smart contract graphs. It captures long-range dependencies that are critical for identifying vulnerabilities arising from complex function call relationships within the contract. By combining structural information with rich semantic content,

ExploreFormer addresses the challenge of previous methods that struggled to fully exploit the global context, allowing it to identify key vulnerability-specific nodes and aggregate global information across the contract graph for more accurate detection.

ConSense is evaluated on the IR-ESCD dataset, a benchmark that includes eight distinct vulnerability types found in real-world smart contracts, allowing for a comprehensive assessment of its performance. The framework shows optimal detection results for seven vulnerabilities, demonstrating impressive accuracy across different eight types. For timestamp dependency, it detects vulnerabilities with an accuracy of 94.34%, while for block number dependency, the accuracy is 96.96%. Dangerous delegatecall is detected with an accuracy of 98.53%, and Ether frozen at 95.20%. For unchecked external calls, ConSense excels with a detection accuracy of 99.13%, while reentrancy and dangerous Ether strict equality are detected with accuracies of 99.73% and 99.83%, respectively. For integer overflow, while ConSense does not achieve the best performance among the comparison methods, it still delivers a strong detection accuracy of 98.18%, falling only 0.61% short of the highest reported accuracy for this vulnerability. Despite this small gap, the framework's overall performance remains highly competitive across the majority of vulnerability types. In addition to its high accuracy, ConSense demonstrates impressive efficiency. It averages 0.648 seconds per contract for detecting vulnerabilities, making it not only accurate but also fast enough to meet the demands of real-time or large-scale smart contract vulnerability detection in blockchain environments. This rapid detection capability ensures that ConSense addresses the need for both speed and accuracy in safeguarding smart contracts against potential security threats.

In summary, this paper makes the following contributions:

- The proposed novel contract sensing framework, ConSense, addresses key challenges in smart contract vulnerability detection by preserving both structural and semantic information within contract graphs. By comprising two core components, the smart contract graph generator and ExploreFormer, ConSense overcomes the limitations of traditional GNNs by capturing long-range dependencies and global contextual relationships through advanced graph learning techniques.
- The smart contract graph generator is designed to capture inter-function call relationships, representing the global

structure of smart contracts. To overcome the limitations of traditional node representations, the pretrained large-scale language model is employed to extract and embed semantic information into the graph nodes, enhancing the expressiveness and accuracy of the model.

- The structure-aware graph neural network, ExploreFormer, is developed to tackle the challenge of integrating both local function call relationships and global contextual information within smart contracts. By leveraging advanced attention mechanisms, ExploreFormer enables the learning of high-quality contract representations, addressing the inherent complexity of smart contract graphs and improving vulnerability detection.
- Extensive experiments are conducted on a dataset comprising eight types of vulnerabilities in real-world smart contracts. The results demonstrate that ConSense outperforms existing methods in terms of detection accuracy while also achieving rapid detection speeds, addressing the challenge of balancing computational efficiency with detection precision.

## II. RELATED WORK

### A. Rule-Based Vulnerability Detection Approaches

*Symbolic Execution* approaches [7], [8], [9], [27] abstract contract inputs as symbolic values and employs constraint solving to identify vulnerabilities along execution paths. Oyente [7] utilizes symbolic execution technology to retrieve critical paths from control flow graphs and identifies contract vulnerabilities through constraint solving. Osiris [27] integrates symbolic execution with taint analysis to uncover integer bugs in smart contracts. The primary limitation of symbolic execution lies in its path explosion problem [28], leading to performance degradation when dealing with complex contracts.

*Formal Verification* tools [10], [11], [12] detect potential vulnerabilities by verifying whether smart contracts meet expected security and correctness requirements. Securify [10] achieves this by designing compliance and violation patterns to prove contract correctness and identify violations. ZEUS [11] leverages symbolic model checking techniques and constrained Horn clauses to swiftly verify contract security. The main limitation of formal verification is its requirement for highly specialized knowledge, resulting in substantial initial costs.

*Static Analysis* approaches [13], [14], [15] do not require executing the contract but rely on predefined security rules to identify potential vulnerabilities. Slither [14] transforms input contracts into an intermediate representation for analyzing security issues. Clairvoyance [15] is specifically designed to detect reentrancy vulnerabilities and proposes five path protection techniques to reduce false positives. A significant issue with these static analysis tools is their heavy reliance on complex prior rules defined by experts, which limits their accuracy and scalability.

*Fuzz Testing* takes a dynamic approach, executing smart contracts by injecting large volumes of random or anomalous inputs and monitoring the resulting behavior for signs of vulnerabilities [29]. ILF [30] improves the efficiency of fuzz testing by utilizing imitation learning to derive high-quality inputs from

symbolic execution, allowing the fuzzer to better explore critical execution paths with increased speed and coverage. Tools like sFuzz [18] and IR-Fuzz [17] further optimize this process by refining their sequence generation strategies, allowing them to explore a broader range of contract states, including those hidden in rarely triggered branches of the execution path. Fuzz testing's dynamic nature provides broader vulnerability detection coverage compared to static methods, but this comes at the cost of high computational overhead [31]. Exploring the vast state spaces of smart contracts demands significant computational resources and extended time, making fuzz testing less efficient regarding time-to-detection, especially when dealing with complex contracts or large-scale systems.

### B. Deep Learning-Based Vulnerability Detection Approaches

1). *Text Feature-based* approaches focus on extracting key textual information from divergent representations of smart contracts, such as source code, opcode, or bytecode [32], [33]. They primarily utilize sequence models like LSTM and Gated Recurrent Units (GRU [34]), which automatically learn vulnerability patterns from the sequential nature of contract data, offering a more efficient alternative to traditional rule-based approaches. SaferSC [19], one of the earliest deep learning models in this domain, leverages LSTM to analyze smart contract opcodes, enabling the automatic learning of contract features and improving the detection of vulnerabilities. ContractWard [35] refines this approach by extracting binary features from simplified operation codes, constructing detection models using five different machine learning algorithms (including decision trees, random forests, and support vector machines) and two sampling algorithms to increase detection robustness. In terms of enhancing feature learning, ReChecker [21] focuses on smart contract source code and employs a bidirectional LSTM coupled with an attention mechanism to specifically detect reentrancy vulnerabilities, which are notoriously difficult to identify with traditional methods. This approach captures dependencies in both forward and backward directions within the code sequence, while the attention mechanism selectively focuses on the most critical parts of the contract for vulnerability detection. On the other hand, ESCORT [36] utilizes GRU to build a feature extractor that captures semantic and syntactic information from smart contract bytecode, thereby improving the model's ability to generalize across different contract structures. In addition, recent studies [37], [38] have investigated the application of LLMs to enhance smart contract security, revealing their potential to further improve vulnerability detection and mitigation in smart contract systems.

Despite their advantages, the text-based approaches treat smart contracts primarily as sequences of textual data, overlooking the structural properties inherent in code, such as control flow and data flow. As a result, they fail to fully capture the complex interaction patterns between different functions and variables within smart contracts, limiting their ability to detect vulnerabilities that arise from intricate execution paths or dependencies. This omission reduces the overall effectiveness in accurately detecting certain types of vulnerabilities, particularly



those requiring a more comprehensive understanding of contract structure and behavior.

2). *GNN-based* approaches [23], [25], [39], [40] have gained significant attention in smart contract vulnerability detection due to their ability to capture the structural properties of contracts, which are often overlooked in traditional and text-based methods. They convert smart contracts into graph structures where nodes represent various program elements, such as functions and variables, and edges represent their interactions, such as function calls or data dependencies [41], [42]. Once the contract is represented as a graph, GNNs are employed to learn the underlying patterns and vulnerabilities by propagating information through the nodes and aggregating features from local neighborhoods. For example, TMP [39] constructs contract graphs from the source code, assigning different node types to program elements like functions, variables, and operations. It employs a normalization phase to highlight the key nodes in the contract graph and uses a temporal message propagation network to analyze the dynamic flow of information within the contract, effectively capturing time-sensitive vulnerabilities. By leveraging GNNs, TMP successfully models the intricate relationships between contract elements, but it still relies heavily on structural information and overlooks deeper semantic features.

Further advancements have been made by methods like AME [23] and slice joint graphs [43], which aim to improve the interpretability and accuracy of GNN-based models. AME builds upon TMP by incorporating expert knowledge using an attention-based multi-encoder network, which fuses graph features with predefined expert patterns, allowing the model to more accurately capture complex vulnerabilities with human interpretable results. Similarly, Cai et al. [43] enhance contract graph construction by representing contract functions as slice joint graphs, combining multiple graph representations to capture different aspects of contract behavior. A bidirectional gated GNN is then used to analyze these graphs and detect vulnerabilities by considering both forward and backward dependencies in the contract's function call graph. Additionally, DMT [24] enhances detection performance by transforming both source code and bytecode into graph structures and applying mutual learning between these two contract graph forms. Despite these advances, these methods still face limitations in fully capturing the rich semantic information embedded in contract statements and struggle with aggregating global contextual information across the entire contract graph. To address these shortcomings, ConSense is proposed as a comprehensive solution that emphasizes both semantic extraction and global information modeling, leading to improved accuracy in smart contract vulnerability detection.

### III. METHODS

#### A. Framework

As shown in Fig. 1, ConSense comprises two core components: the SC Graph Generator and ExploreFormer. The former constructs contract graphs while retaining both structural and semantic information from function call relationships, while the latter analyzes these graphs by integrating local node

information with global context through attention mechanisms for effective vulnerability detection.

#### B. Smart Contract Graph Generator

The SC Graph Generator addresses the challenge of information loss often encountered in traditional GNN-based approaches, which frequently omit subordinate nodes during graph construction, resulting in incomplete representations of smart contracts. To overcome this limitation, the SC Graph Generator adopts a holistic approach that ensures the inclusion of all contract elements and accurately reflects the call relationships among functions. As illustrated in Fig. 2, this generator comprises two parallel branches: the Graph Structure Construction (GSC) described in Section III-B1, which meticulously constructs the contract graph by capturing its structural components, and the Node Representation Extraction (NRE) described in Section III-B2, which employs LLMs to convey deep semantic information from contract functions. By effectively integrating both structural and semantic information, the SC Graph Generator produces a comprehensive Smart Contract Graph, providing a solid foundation for subsequent vulnerability detection analyses.

Fig. 2 illustrates the detailed workflow of the SC Graph Generator. The process starts with the raw smart contract source code, which is parsed into an Abstract Syntax Tree (AST) using the Solidity compiler. From this AST, a call graph is generated to capture the structural relationships between the contract's functions. This call graph is then encoded into an adjacency matrix, forming the topological structure of the smart contract graph. Simultaneously, the raw contract data for each node is discretized and processed through Code Llama to extract high-level semantic features. These features are then used to construct a representation matrix for all nodes in the graph. The final graph structure and node representations provide the necessary foundation for the subsequent vulnerability detection tasks.

1) *Graph Structure Construction*: The contract graph is formally defined as  $G(V, E)$ , where  $V$  represents the set of nodes and  $E$  denotes the set of directed edges. The nodes in this graph, denoted as  $V = \{v_1, v_2, v_3, \dots, v_n\}$ , correspond to the functions within the smart contract, with  $n$  indicating the total number of nodes. To capture the diverse functionalities of smart contracts, the nodes are categorized into three distinct types: The first type encompasses constructors, modifier functions, and custom functions, which can be directly extracted as code blocks from the contract source code. These functions play a crucial role in defining the contract's behavior and are essential for its operational integrity. The second type consists of global variable functions, which are derived by consolidating the declaration statements of the contract's global variables. These functions facilitate the management of state variables and contribute to the overall functionality of the contract. The third type categorizes the fallback function, which is particularly noteworthy due to its association with numerous vulnerabilities that can arise in smart contracts. Each of these node types is equally significant, as they collectively encapsulate the complete semantic information contained within the contract.

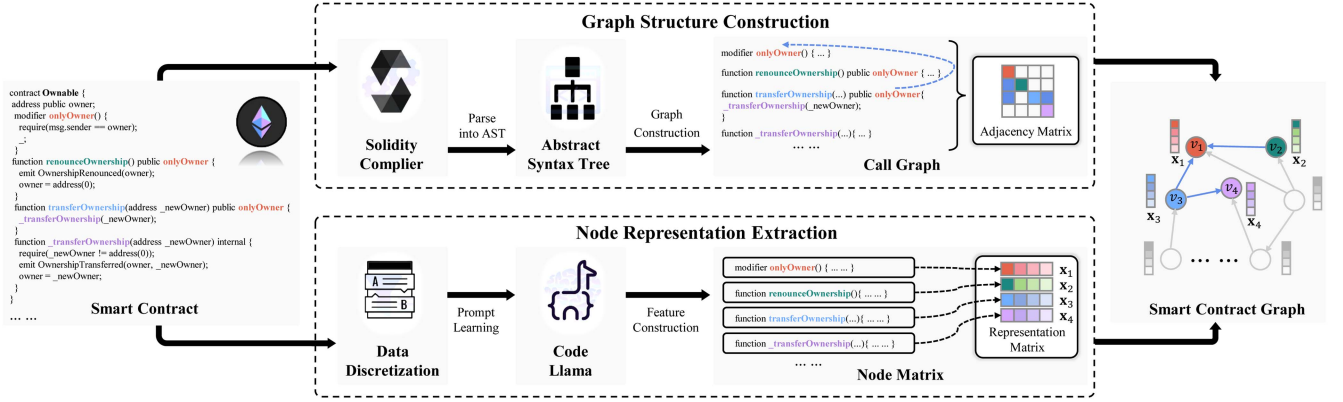


Fig. 2. Architecture of the Smart Contract Graph Generator, comprising two parallel branches: (Top) Graph Structure Construction, which models the contract’s call hierarchy by extracting nodes and edges from function dependencies; (Bottom) Node Representation Extraction, which encodes both structural topology and semantic features into unified node embeddings for subsequent analysis.

The directed edges  $E = \{e_1, e_2, e_3, \dots, e_m\}$ , where  $m$  represents the total number of edges, connect the different nodes and represent the calls and interactions between various functions. These edges reflect the data flow and control flow within the contract, with their direction indicating the order of function execution or jumps. By modeling the relationships among functions in this manner, the contract graph provides a clear representation of the interdependencies between functions, facilitating the identification of potential vulnerabilities and highlighting critical paths and interactions that may lead to security issues. The final adjacency matrix of the contract graph is constructed by utilizing the static analysis tool Slither [14] to generate an abstract syntax tree from the source code, producing a call graph with the *print call-graph* command.

2) *Node Representation Extraction*: Following the construction of the graph structure by the GSC, the NRE is tasked with providing deep semantic representations for each node in parallel. The semantic information of contract functions plays a crucial role in accurately capturing the functional logic embedded within the contract graph. To effectively leverage LLMs in this context, we utilize the pre-trained Code Llama [44]. While Code Llama demonstrates robust capabilities in general programming tasks, its training does not encompass the specific semantics and security nuances of smart contracts. To enhance its domain-specific understanding, we curated and preprocessed three smart contract datasets from [17], [22], [45], removing duplicates to compile 91,119 unique smart contract source code files. These files are then used to fine-tune Code Llama, enabling it to better capture the distinct patterns and features inherent to smart contract code. This fine-tuning process enhances Code Llama’s ability to represent contract statements more effectively, thereby improving its capacity for vulnerability detection.

In the *Node Representation Extractor (NRE)*, the smart contract graph  $G(V, E)$  is transformed into a feature matrix  $H \in \mathbb{R}^{n \times d}$ , where  $n$  represents the number of nodes, and  $d$  denotes the dimension of each node’s feature vector  $h_i \in H$ . Each node corresponds to a distinct function or element within the smart contract, encapsulating critical aspects of the contract’s execution logic. Traditional methods for generating node features

often rely solely on node attributes or adjacency relationships, which do not adequately capture the rich semantic information embedded within the contract’s source code. To address this limitation, we leverage unsupervised prompt learning with Code Llama, a pre-trained large language model, to enrich the node features with deeper semantic insights. During this process, an input sequence of contract code  $S = [w_1, w_2, \dots, w_n]$  is provided, where random tokens  $w_i$  are masked to create  $S_{\text{masked}}$ . Code Llama is tasked with predicting the masked token  $w_i$  based on the context provided by the unmasked tokens in  $S_{\text{masked}}$ , enabling the model to learn both functional and logical relationships within the contract’s code. By doing so, Code Llama generates a  $d$ -dimensional feature vector for each node that encodes the semantic content of the corresponding function or statement block. As illustrated in Fig. 2. The resulting node feature matrix  $H$ , along with the adjacency matrix obtained from the GSC, is then used as input for ExploreFormer, allowing it to integrate both structural and semantic information on the smart contract graph and thereby improving the model’s capacity for vulnerability detection in smart contracts.

### C. ExploreFormer

Traditional GNN-based approaches, which aggregate features from neighboring nodes into a central node, often struggle to highlight critical nodes affecting contract security and to capture the global context in complex graph structures, such as those involving iterative loops and multiple function calls [39], [46]. To address these limitations, ExploreFormer, a structure-aware graph learning module, is designed to effectively mine both the semantic information and structural relationships within smart contracts to identify potential security vulnerabilities. By extracting subgraphs around key nodes, ExploreFormer enables a more comprehensive capture of security-relevant features. As illustrated in Fig. 3, the process unfolds in three stages: (1) Local Feature Fusion, where an attention mechanism encodes local subgraphs to emphasize nodes related to specific vulnerabilities; (2) Global Information Exchange, which leverages global attention to capture long-range dependencies between nodes,

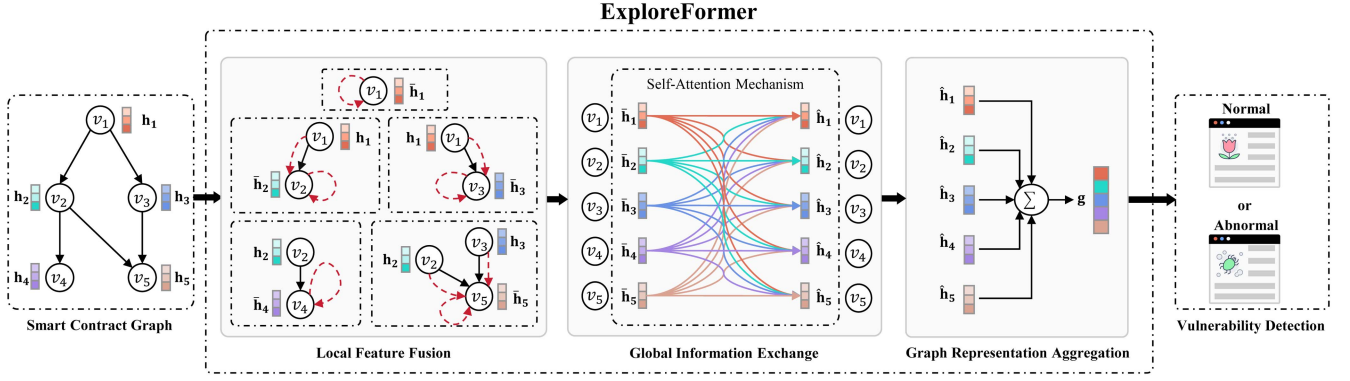


Fig. 3. The ExploreFormer architecture processes smart contracts through three hierarchical stages: (1) Local Feature Fusion employs attention mechanisms to encode vulnerability-sensitive patterns within subgraph structures; (2) Global Information Exchange establishes cross-graph node dependencies via transformer-based attention; and (3) Graph Representation Aggregation synthesizes node-level features into a holistic graph embedding optimized for vulnerability detection.

integrating broader contextual information across the graph; and (3) Graph Representation Aggregation, where node features are pooled into a complete graph representation, improving accuracy in vulnerability detection. This combination of local and global perspectives allows ExploreFormer to more effectively detect complex vulnerabilities in smart contracts.

1) *Local Feature Fusion*: The Local Feature Fusion stage in ExploreFormer aims to capture the local structural and semantic relationships surrounding key nodes in the smart contract graph. By focusing on  $k$ -hop neighborhoods around each node, this stage is designed to highlight the critical neighboring nodes that contribute to security vulnerabilities. Rather than relying on global graph aggregation, which can dilute the significance of local structures, Local Feature Fusion isolates relevant subgraphs to emphasize the most important contract elements for vulnerability detection.

For the central function node  $v_i$  in the contract graph  $G(V, E, H)$ , ExploreFormer extracts a  $k$ -hop subgraph  $S(V_k, E_k, H_k)$ , where  $V_k$ ,  $E_k$ , and  $H_k$  represent the nodes, edges, and feature matrix of the subgraph, respectively. The neighborhood of node  $v_i$  is defined as  $\mathcal{N}(v_i) = \{v_j \mid d_G(i, j) \leq k\}$ , where  $d_G(i, j) \leq k$  indicates that the distance between neighboring nodes  $v_i$  and  $v_j$  is less than or equal to  $k$ . This subgraph extraction allows ExploreFormer to focus on a localized set of nodes and edges that are crucial for understanding the security-relevant properties of the contract. After forming the subgraph, a self-attention mechanism is applied to capture the influence of neighboring nodes on the central node in (1):

$$e_{ij}^{(l)} = \text{LeakyRelu} \left( \mathbf{W}_e^{(l)} \cdot [\mathbf{h}_i^{(l)} \parallel \mathbf{h}_j^{(l)}] \right) \quad (1)$$

Where  $\mathbf{h}_i^{(l)}$  and  $\mathbf{h}_j^{(l)}$  denote the hidden states of nodes  $v_i$  and  $v_j$  at layer  $l$ , and  $[\cdot \parallel \cdot]$  represents the concatenation of their hidden states. The parameter matrix  $\mathbf{W}_e^{(l)}$  applies a linear transformation, and the LeakyRelu activation function introduces nonlinearity, enabling the model to calculate the contribution of node  $v_j$  to the central node  $v_i$ . This attention mechanism ensures that ExploreFormer can accurately assess the importance of local node interactions, which is essential for identifying security vulnerabilities within the contract.

In the latter part of the Local Feature Fusion stage, once the raw attention scores between the central node  $v_i$  and its neighboring nodes  $v_j$  are calculated, they are normalized using the Softmax function to ensure proper differentiation between the internal elements of the contract graph. This normalization, expressed as  $\alpha_{ij}^{(l)}$ , assigns weights to the neighboring nodes in (2), reflecting their importance concerning the central node.

$$\alpha_{ij}^{(l)} = \text{Softmax} \left( e_{ij}^{(l)} \right) = \frac{\exp \left( e_{ij}^{(l)} \right)}{\sum_{v_x \in \mathcal{N}(v_i) \cup \{v_i\}} \exp \left( e_{ix}^{(l)} \right)} \quad (2)$$

where  $\mathcal{N}(v_i)$  denotes the neighboring nodes of  $v_i$ . After obtaining these normalized attention scores, the feature representation of the central node is updated by aggregating the contextual information from both the central node and its neighbors. This update process is carried out through a linear transformation with the parameter matrix  $\mathbf{W}_\alpha^{(l)}$ , followed by a ReLU activation function in (3).

$$\bar{\mathbf{h}}_i^{(l+1)} = \text{Relu} \left( \alpha_{ii}^{(l)} \cdot \mathbf{W}_\alpha^{(l)} \cdot \mathbf{h}_i^{(l)} + \sum_{v_j \in \mathcal{N}(v_i)} \alpha_{ij}^{(l)} \cdot \mathbf{W}_\alpha^{(l)} \cdot \mathbf{h}_j^{(l)} \right) \quad (3)$$

This process allows for the effective fusion of local subgraph information, ensuring that the central node's features are updated to reflect the most relevant and influential neighboring nodes.

2) *Global Information Exchange*: The second stage of ExploreFormer, Global Information Exchange, is designed to overcome the limitations of local feature aggregation by capturing long-range dependencies across the entire smart contract graph. Although the local self-attention mechanism effectively models the interactions within a limited neighborhood, it struggles to capture broader global contexts, which is crucial for identifying complex vulnerabilities distributed across various functions in a contract. To address this challenge, ExploreFormer employs a global self-attention mechanism that disregards the original graph structure and treats the graph as multiple sets of node pairs. This allows for the modeling of interactions between all nodes, enabling the effective fusion of global contextual information.

In this stage, the representation of each node  $\mathbf{h}_u^{(l+1)}$  is updated by considering the features of all other nodes  $\mathbf{h}_v^{(l+1)}$  in the graph. The update process is governed by a weighted sum in (4), where the contribution of each node  $\mathbf{h}_v^{(l+1)}$  to the target node  $\mathbf{h}_u^{(l+1)}$  is determined by an exponential kernel  $\kappa_{\text{exp}}$ .

$$\mathbf{h}_u^{(l+1)} = \sum_{v_u \in V} \frac{\kappa_{\text{exp}}(\hat{\mathbf{h}}_u^{(l+1)}, \hat{\mathbf{h}}_v^{(l+1)})}{\sum_{v_w \in V} \kappa_{\text{exp}}(\hat{\mathbf{h}}_u^{(l+1)}, \hat{\mathbf{h}}_w^{(l+1)})} f(\hat{\mathbf{h}}_v) \quad (4)$$

where  $f(\hat{\mathbf{h}}_v) = \mathbf{W}_V \mathbf{h}$  is a linear transformation of the node feature vector, and the kernel  $\kappa_{\text{exp}}$  is parameterized by learned weight matrices  $\mathbf{W}_Q$  and  $\mathbf{W}_K$  in (5). This exponential kernel computes the similarity between node features and is critical for determining how much influence one node should have on another during the information exchange process.

$$\kappa_{\text{exp}}(\mathbf{h}, \mathbf{h}') := \exp\left(\langle \mathbf{W}_Q \mathbf{h}, \mathbf{W}_K \mathbf{h}' \rangle / \sqrt{d_{\text{out}}}\right) \quad (5)$$

where  $d_{\text{out}}$  is the output dimension of the feature vector at layer  $l + 1$ . The dot product operation captures the similarity between node features, and the kernel emphasizes nodes with high feature similarity, allowing the model to aggregate global information effectively. This mechanism enables ExploreFormer to model complex dependencies and interactions across the whole contract graph, ensuring that critical security-related features are captured and represented globally.

3) *Graph Representation Aggregation*: In the third stage of ExploreFormer, *Graph Representation Aggregation*, the goal is to integrate the node representations, which were refined through both local feature fusion and global information exchange, into a comprehensive graph-level representation. This stage is crucial for providing a global understanding of the smart contract graph, facilitating accurate vulnerability detection.

The node representations  $\mathbf{h}_u^{(l+1)}$  from the previous layer are aggregated using a *graph pooling* operation. Specifically, for each node  $v_u \in V$  in the graph, the feature vectors  $\mathbf{h}_u^{(l+1)}$  are summed to generate a unified graph representation  $\mathbf{g}$  in (6).

$$\mathbf{g} = \sum_{v_u \in V} \mathbf{h}_u^{(l+1)} \quad (6)$$

where  $\mathbf{h}_u^{(l+1)}$  denotes the node representation at layer  $l + 1$ , and the sum is taken over all nodes  $v_u$  in the graph  $V$ . This pooling operation captures the combined influence of all nodes and their interactions, summarizing the entire contract graph into a single vector  $\mathbf{g}$  that encapsulates the structural and semantic information of the contract.

At the end of the ExploreFormer, the graph representation  $\mathbf{g}$  is passed through a *simple linear layer* to produce the final output, which represents the results of the vulnerability detection task. This step ensures that the aggregated node-level information is transformed into a form that can be effectively utilized for downstream tasks, such as identifying security vulnerabilities. The progressive combination of the three stages—local feature learning, global information exchange, and graph representation aggregation—provides a comprehensive understanding of the

TABLE I  
DETAILS OF THE IR-ESCD DATASET

| Vulnerability | BN   | DE  | EF  | SE   | OF   | RE   | TP   | UC   |
|---------------|------|-----|-----|------|------|------|------|------|
| Total         | 1804 | 401 | 408 | 1988 | 2515 | 4095 | 2838 | 3850 |
| Vulnerable    | 128  | 75  | 84  | 50   | 42   | 119  | 310  | 115  |

TABLE II  
DETAILS OF THE SCVHUNTER-SCD DATASET

| Vulnerability | Reentrancy | Timestamp | Transaction State |
|---------------|------------|-----------|-------------------|
| Total         | 300        | 104       | 300               |
| Vulnerable    | 203        | 60        | 180               |

contract's structure, significantly enhancing the model's ability to handle complex graph structures in smart contracts.

#### D. Loss Function

In smart contract vulnerability detection, the ConSense is trained to perform a binary classification task, determining whether a given contract graph contains a specific type of security vulnerability. The cross-entropy function optimizes the ExploreFormer during the training phase in (7).

$$L(y, \hat{y}) = - \sum_{i=1}^N y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i) \quad (7)$$

where  $L$  represents the total loss, while  $N$  is the number of contract graph samples in a single batch of input. For each sample  $i$ ,  $y_i$  is the ground truth, which can either be 0 (indicating no vulnerability) or 1 (indicating the presence of a vulnerability).  $\hat{y}_i$  is the predicted probability that the model assigns to the sample containing a vulnerability.

## IV. EXPERIMENTS

### A. Datasets

*IR-ESCD* [17] comprises 12,515 smart contracts sourced from the Ethereum mainnet,<sup>1</sup> with each contract averaging 80 lines of code. It focuses on eight types of security vulnerabilities: timestamp dependency (TP), block number dependency (BN), dangerous delegatecall (DE), Ether frozen (EF), unchecked external call (UC), reentrancy (RE), integer overflow (OF), and dangerous Ether strict equality (SE). Table I presents the distribution of each vulnerability type in the dataset, including the total number of associated smart contracts and the count of vulnerable contracts.

*SCVHunter-SCD* [25] is derived from the Ethereum Open Dataset [45], with each contract containing an average of 581 lines of code. As detailed in Table II, the dataset includes 203 contracts with reentrancy vulnerabilities, 60 contracts exhibiting timestamp dependency vulnerabilities, and 180 contracts with transaction state dependency vulnerabilities. In contrast to the IR-ESCD dataset, this dataset offers a more balanced distribution of samples across different vulnerability types, facilitating a more comprehensive evaluation of the robustness

<sup>1</sup>Etherscan: <https://etherscan.io/>



and adaptability of our approach across various vulnerability scenarios. This balance contributes to enhancing the reliability and generalizability of the experimental results.

### B. Baseline Methods

1) *Rule-Based Vulnerability Detection Approaches*: In the conducted experiments, six rule-based vulnerability detection approaches are compared with ConSense's performance. Mythril [8] and Oyente [7] are static analysis tools that leverage symbolic execution to identify vulnerabilities in smart contracts by exploring multiple execution paths. Securify [10] utilizes a rule-matching approach to detect security flaws by comparing the contract code with predefined patterns. Fuzzing-based methods are also considered, such as ILF [30], which combines learning techniques with symbolic execution imitation, and sFuzz [18], a feedback-guided adaptive fuzzer. Additionally, IR-Fuzz [17], an extension of sFuzz, improves seed generation and branching strategies for more effective vulnerability detection. These approaches are selected to provide a comprehensive benchmark, covering a range of static analysis, symbolic execution, and fuzzing-based methods for detecting vulnerabilities in smart contracts.

2) *Deep Learning-Based Vulnerability Detection Approaches*: For a comprehensive evaluation of deep learning-based vulnerability detection, four widely recognized open-source methods are selected due to their effectiveness in analyzing smart contract security. SafeSC [19], an LSTM-based model, focuses on contract opcodes to detect vulnerabilities, making it a valuable benchmark for opcode analysis. ReChecker [21], employing a BLSTM-ATT model, concentrates on source code, offering insights into sequence detection performance. TMP [22], a temporal message propagation network, learns features from normalized contract graphs, representing a powerful approach for leveraging graph structures. AME [23] builds upon TMP by integrating expert knowledge vectors with graph-based learning for enhanced vulnerability detection. SCVHunter [25], a graph-based smart contract vulnerability detection method, uses heterogeneous graph neural networks to achieve leading performance. These methods are chosen because they represent a range of advanced deep learning techniques, covering both text and graph feature extraction. Additionally, GCN [47] is incorporated as a baseline to compare its performance against ExploreFormer in the ConSense, with all other components remaining consistent to ensure a fair comparison.

### C. Evaluation Metric

Smart contract vulnerability detection is framed as a binary classification problem, where contracts are categorized as either vulnerable (1) or non-vulnerable (0). The performance of detection methods is evaluated using four standard metrics: accuracy, precision, recall, and F1 score. These metrics are computed based on the True Positive (TPo), False Positive (FP), True Negative (TN), and False Negative (FN) values, with their respective definitions provided below.

*Accuracy*: The proportion of correctly identified smart contracts, both vulnerable and non-vulnerable, to the total number

TABLE III  
MODEL AND TRAINING HYPERPARAMS

| Hyperparameter | Value | Hyperparameter | Value |
|----------------|-------|----------------|-------|
| Subgraph hops  | 2     | Hidden size    | 512   |
| Batch size     | 32    | Train epochs   | 300   |
| Learning rate  | 3e-4  | Weight decay   | 1e-4  |

of contracts in the dataset.

$$\text{Acc} = \frac{TPo + TN}{TPo + TN + FP + FN} \quad (8)$$

*Precision*: The ratio of smart contracts correctly classified as vulnerable (TPo) to the total number of contracts predicted as vulnerable, reflecting the correctness of positive vulnerability predictions.

$$\text{Precision} = \frac{TPo}{TPo + FP} \quad (9)$$

*Recall*: The ratio of smart contracts correctly classified as vulnerable (TPo) to the total number of actual vulnerable contracts, indicating the model's ability to identify all vulnerable contracts.

$$\text{Recall} = \frac{TPo}{TPo + FN} \quad (10)$$

*F1 score*: The harmonic mean of precision and recall, providing a balanced measure of the model's performance in detecting vulnerabilities while minimizing both false positives and false negatives.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (11)$$

### D. Implementation Details

ConSense is implemented using the PyTorch framework [48] and the Torch Geometric library [49], with the subgraph hop count  $k$  set to 2 and the hidden layer size set to 512. The model is trained for 300 epochs with a fixed batch size of 32 and an initial learning rate of 0.0003, using the AdamW optimizer [50] with a weight decay of  $1 \times 10^{-4}$ . Experiments are conducted on hardware with an Intel Xeon Gold 6348 CPU at 2.6GHz and an NVIDIA GeForce RTX 3090 GPU with 24GB memory. The dataset is divided into training, validation, and test sets in a ratio of 7:1:2, with each method being repeated five times to ensure reliability, and average detection results are reported. Table III summarizes the model architecture hyperparameters and training settings used in our experiments.

### E. Results

1) *Performance Comparison on the IR-ESCD*: Table IV provides a comprehensive accuracy comparison of smart contract vulnerability detection approaches across eight different types of security vulnerabilities: timestamp dependency (TP), block number dependency (BN), dangerous delegatecall (DE), Ether frozen (EF), unchecked external call (UC), reentrancy (RE), integer overflow (OF), and dangerous Ether strict equality (SE).



TABLE IV  
ACCURACY COMPARISON (%) WITH DIFFERENT APPROACHES OF SMART CONTRACT VULNERABILITY DETECTION ON THE IR-ESCD DATASET

| Methods | RE              | TP                      | OF                      | BN                             | DE                      | EF                      | UC                      | SE                      |
|---------|-----------------|-------------------------|-------------------------|--------------------------------|-------------------------|-------------------------|-------------------------|-------------------------|
| T       | Mythril [8]     | 95.09 <sup>+04.64</sup> | -                       | 89.87 <sup>+08.31</sup>        | 89.97 <sup>+06.99</sup> | 70.95 <sup>+27.58</sup> | -                       | -                       |
| R       | Oyente [7]      | 94.61 <sup>+05.12</sup> | 86.86 <sup>+07.48</sup> | 74.76 <sup>+23.42</sup>        | -                       | -                       | -                       | -                       |
| V       | Securify [10]   | 91.52 <sup>+08.21</sup> | -                       | -                              | -                       | 79.42 <sup>+15.78</sup> | 91.24 <sup>+07.89</sup> | -                       |
| D       | ILF [30]        | -                       | -                       | -                              | 87.53 <sup>+09.43</sup> | 80.99 <sup>+17.54</sup> | 94.71 <sup>+04.42</sup> | -                       |
|         | sFuzz [18]      | 95.20 <sup>+04.53</sup> | 87.25 <sup>+07.09</sup> | 89.98 <sup>+08.20</sup>        | 88.37 <sup>+08.59</sup> | 83.33 <sup>+15.20</sup> | 83.85 <sup>+11.35</sup> | 94.26 <sup>+04.87</sup> |
|         | IR-Fuzz [17]    | 98.77 <sup>+00.96</sup> | 90.25 <sup>+04.09</sup> | <b>98.79</b> <sup>-00.61</sup> | 94.18 <sup>+02.78</sup> | 95.33 <sup>+03.20</sup> | 95.05 <sup>+00.15</sup> | 98.10 <sup>+01.03</sup> |
|         |                 |                         |                         |                                |                         |                         |                         | 99.73 <sup>+00.10</sup> |
| D       | SafeSC [19]     | 45.68 <sup>+54.05</sup> | 52.29 <sup>+42.05</sup> | 46.45 <sup>+51.73</sup>        | 53.08 <sup>+43.88</sup> | 55.00 <sup>+43.53</sup> | 50.85 <sup>+44.35</sup> | 51.73 <sup>+47.40</sup> |
| L       | ReChecker [21]  | 94.76 <sup>+04.97</sup> | 74.92 <sup>+19.42</sup> | 92.11 <sup>+06.07</sup>        | 80.73 <sup>+16.23</sup> | 71.58 <sup>+26.95</sup> | 51.00 <sup>+44.20</sup> | 91.27 <sup>+07.86</sup> |
| V       | TMP [22]        | 28.28 <sup>+71.45</sup> | 57.14 <sup>+37.20</sup> | -                              | -                       | -                       | -                       | -                       |
| D       | AME [23]        | 93.94 <sup>+05.79</sup> | 63.49 <sup>+30.85</sup> | -                              | -                       | -                       | -                       | -                       |
|         | ConSense†       | 99.59 <sup>+00.14</sup> | 93.40 <sup>+00.94</sup> | 98.18 <sup>+00.00</sup>        | 96.20 <sup>+00.76</sup> | 97.06 <sup>+01.47</sup> | 91.30 <sup>+03.90</sup> | 98.84 <sup>+00.29</sup> |
|         | <b>ConSense</b> | <b>99.73</b>            | <b>94.34</b>            | 98.18                          | <b>96.96</b>            | <b>98.53</b>            | <b>95.20</b>            | <b>99.13</b>            |
|         |                 |                         |                         |                                |                         |                         |                         | <b>99.83</b>            |

‘-’ denotes that a tool fails to detect this type of vulnerability. ConSense† employs the graph convolution network instead of ExploreFormer. TRVD means traditional rule-based vulnerability detection approaches, while DLVD represents deep learning-based ones. The increase and loss of metrics are recorded in red and green in this table.

*ConSense* achieves high accuracy across all types of vulnerabilities, including 99.73% for RE, 96.20% for BN, 99.13% for UC, and 99.83% for SE. By capturing contract semantic information, *ConSense* accurately identifies vulnerabilities associated with variables, such as BN and SE. Its excellent performance on vulnerability types related to function calls, such as RE and UC, is attributed to *ConSense*’s effective modeling of the contract’s structural information. In comparison, *ConSense*†, which replaces the ExploreFormer with a traditional graph convolution network (GCN), shows slightly reduced performance, most notably in TP (93.40%), EF (91.30%), and RE (99.59%). This demonstrates how ExploreFormer contributes to enhancing the modeling of both local and global graph features, leading to improved vulnerability detection compared to GCN-based *ConSense*†.

When comparing *ConSense* with traditional rule-based vulnerability detection methods, significant differences can be observed in accuracy. For example, *ConSense* reaches 94.34% for timestamp dependency, outperforming Oyente, which achieves 86.86%. Similarly, *ConSense* achieves 99.13% accuracy for unchecked external calls, while sFuzz records 94.26%. In the case of reentrancy detection, *ConSense* reaches 99.73%, significantly higher than ILF’s 95.20%. However, for integer overflow detection, *ConSense*’s accuracy of 98.18% is slightly lower by 0.61% compared to IR-Fuzz, which achieves 98.79%. While rule-based methods can still excel in some cases, *ConSense* consistently outperforms these methods in the majority of vulnerability types.

In contrast to deep learning-based approaches, *ConSense* demonstrates noticeable performance improvements. SafeSC and ReChecker, both based on text features, achieve 55.00% and 71.58% accuracy respectively for dangerous delegatecall detection, whereas *ConSense* achieves 98.53%. The GNN-based approaches, TMP and AME, which leverage graph-based techniques, also exhibit lower performance in key vulnerability types. For instance, TMP achieves 92.11% accuracy for integer overflow detection and AME reaches 98.18%, but neither method surpasses *ConSense*’s results. This consistent performance across all vulnerability types highlights the effectiveness of *ConSense*’s architecture, particularly in addressing complex

patterns that are essential for handling vulnerabilities such as block number dependency and reentrancy, where *ConSense* achieves near-perfect accuracy.

2) *Performance Comparison on the SCVHunter-SCD*: Table V demonstrates the performance evaluation of various smart contract vulnerability detection methods on the SCVHunter-SCD dataset, focusing on three critical vulnerabilities: reentrancy, timestamp dependency, and transaction state dependency. The comparison includes *ConSense* and four rule-based traditional tools: Oyente [7], ContractFuzzer [16], Mythril [8], and Slither [14]. *ConSense* achieves significant improvements over these rule-based tools across all three vulnerability types. Specifically, in reentrancy detection, *ConSense* outperforms Slither, the state-of-the-art tool, with an accuracy of 88.33% and an F1 score of 91.14%, marking improvements of 5.66% in accuracy and 3.12% in F1 score. Furthermore, in detecting timestamp dependency and transaction state dependency, *ConSense* surpasses Mythril, the most effective baseline tool, by 9.09% and 14.83% in accuracy, and 22.52% and 42.26% in F1 score, respectively.

Building on these findings, the comparison extends to deep learning-based vulnerability detection approaches, as shown in Table V, where *ConSense* is evaluated alongside five other methods: LSTM [20], GRU [34], TMP [22], AME [23], and SCVHunter [25]. The experimental results highlight *ConSense*’s overall effectiveness in detecting the three critical vulnerabilities, showing competitive performance relative to SCVHunter, the current state-of-the-art method, across Recall, Precision, and F1 score metrics. Specifically, *ConSense* achieves F1 scores of 91.14%, 86.96%, and 92.01% for reentrancy, timestamp dependency, and transaction state dependency, respectively, with improvements of 1.43%, 1.17%, and 8.50%. While SCVHunter reports slightly higher accuracy on the reentrancy and timestamp dependency tasks, its performance can be attributed to its incorporation of vulnerability-specific expert knowledge and the use of a heterogeneous graph neural network architecture, which aids in the extraction of structural patterns tailored to particular vulnerability types.

3) *Efficiency Comparison*: Fig. 4 illustrates the efficiency of various vulnerability detection approaches, focusing on both

TABLE V  
PERFORMANCE COMPARISON (%) WITH DIFFERENT APPROACHES OF SMART CONTRACT VULNERABILITY DETECTION ON THE SCVHUNTER-SCD DATASET

| Methods |                | Reentrancy   |              |              |              | Timestamp Dependency |              |              |              | Transaction State Dependency |              |              |              |
|---------|----------------|--------------|--------------|--------------|--------------|----------------------|--------------|--------------|--------------|------------------------------|--------------|--------------|--------------|
|         |                | Acc          | Recall       | Precision    | F1           | Acc                  | Recall       | Precision    | F1           | Acc                          | Recall       | Precision    | F1           |
| T       | Oyente         | 63.25        | 55.31        | 44.26        | 49.17        | 37.29                | 24.62        | 28.34        | 26.35        | -                            | -            | -            | -            |
| R       | Contractfuzzer | 67.15        | 61.42        | 47.26        | 53.42        | 63.94                | 67.42        | 64.52        | 65.94        | -                            | -            | -            | -            |
| V       | Mythril        | 64.37        | 70.13        | 42.93        | 53.26        | 76.62                | 61.74        | 67.38        | 64.44        | 75.32                        | 43.86        | 57.44        | 49.75        |
| D       | Slither        | 82.67        | 84.09        | 82.68        | 88.02        | 64.79                | 87.67        | 29.79        | 45.65        | 46.05                        | 17.83        | 69.70        | 28.40        |
| D       | LSTM           | 57.68        | 65.93        | 53.42        | 59.02        | 53.44                | 59.17        | 52.12        | 55.42        | 53.47                        | 60.24        | 50.83        | 55.14        |
|         | GRU            | 61.46        | 67.61        | 55.19        | 60.77        | 57.22                | 60.39        | 52.16        | 55.97        | 51.51                        | 63.44        | 53.72        | 58.18        |
| L       | TMP            | 83.57        | 83.05        | 77.17        | 79.99        | 82.39                | 83.13        | 74.65        | 78.66        | -                            | -            | -            | -            |
| V       | AME            | 88.63        | 87.15        | 82.25        | 84.63        | 85.33                | 84.26        | 83.78        | 84.02        | -                            | -            | -            | -            |
| D       | SCVHunetr      | <b>93.72</b> | 90.60        | 88.84        | 89.71        | <b>90.65</b>         | 87.52        | 84.13        | 85.79        | 87.37                        | 83.42        | 83.61        | 83.51        |
|         | ConSense       | 88.33        | <b>90.69</b> | <b>92.31</b> | <b>91.14</b> | 85.71                | <b>91.67</b> | <b>90.91</b> | <b>86.96</b> | <b>90.15</b>                 | <b>93.33</b> | <b>91.00</b> | <b>92.01</b> |

‘-’ denotes that a tool fails to detect this type of vulnerability.

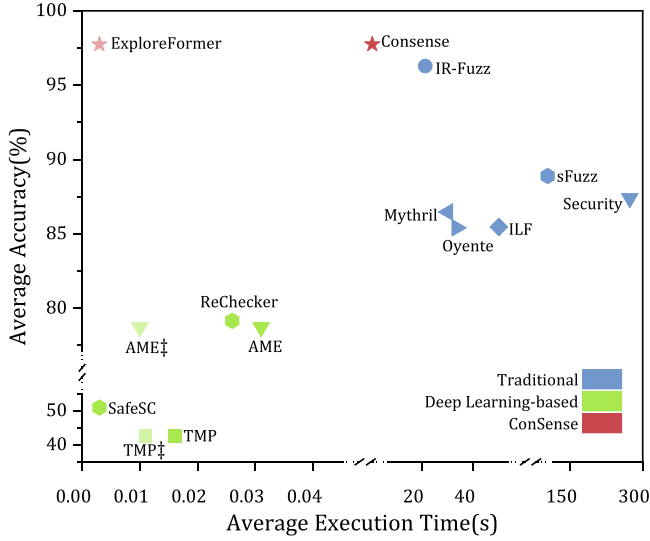


Fig. 4. Visual comparison of efficiency on different vulnerability detection approaches. The horizontal axis represents the average execution time for detecting a single contract, and the vertical axis denotes the average detection accuracy of multiple types of vulnerabilities that can be detected. TMP† and AME† represent the total time cost after TMP and AME remove the time required for contract graph construction.

execution time and average accuracy on all smart contracts. Traditional rule-based approaches, colored in blue, are generally slower than deep learning-based methods, shown in green. For example, Securify takes over 273 seconds, and sFuzz requires 103.52 seconds, making them significantly slower than deep learning models such as AME, which executes in 0.031 seconds. Notably, the slowest deep learning-based method, SafeSC (0.003 seconds), is over 10,000 times faster than the fastest traditional rule-based method, Oyente, which takes 33.50 seconds. This stark contrast highlights the efficiency advantage of deep learning-based approaches in terms of speed.

ConSense achieves a strong balance between speed and average accuracy when compared to other deep learning-based approaches. While methods like TMP and SafeSC are faster, they show a considerable accuracy gap—TMP lags by 55.03% and SafeSC by 46.77% when compared to ConSense’s 97.74% accuracy. This substantial difference demonstrates that, although ConSense takes slightly longer at 0.648 seconds, the trade-off is

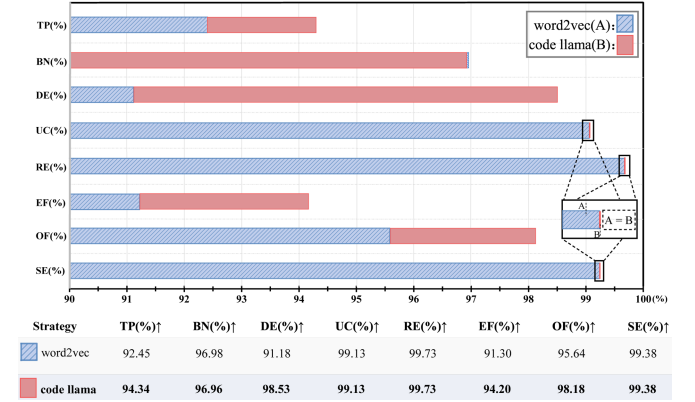


Fig. 5. Accuracy comparison (%) of ConSense with different node representation strategies in smart contract graph generation. ‘A’ represents word2vec, while ‘B’ indicates Code Llama.

more than justified by its significantly higher detection precision. Moreover, the execution time of ConSense’s core component, ExploreFormer, is just 0.003 seconds. The primary time consumption in ConSense occurs during the generation of smart contract graphs which projects the function call relationships within contracts, a step not present in other deep learning-based approaches.

#### F. Ablation Studies

1) *Evaluating the Effectiveness of Different Node Representation Strategies:* Fig. 5 illustrates the accuracy comparison between two different node representation strategies in the smart contract graph generation process: word2vec and Code Llama. Both strategies show strong performance across all eight types of vulnerabilities, but Code Llama consistently delivers higher accuracy in most cases. For instance, in the DE vulnerability detection, Code Llama surpasses word2vec by a margin of 7.35% (98.53% versus 91.18%). Similar trends are observed for EF and OF, with Code Llama achieving 94.20% and 98.18%, respectively, compared to word2vec’s 91.30% and 95.64%. The smallest difference between the two methods occurs in BN and SE, where the accuracy gap is negligible, at 0.02% (96.96% versus 96.98%) and 0.00% (99.38% for both methods). Given these results, ConSense opts for Code Llama as the final node

TABLE VI  
PERFORMANCE COMPARISON (%) OF CONSENSE USING DIFFERENT LOCAL FEATURE FUSION METHODS IN EXPLOREFORMER ACROSS EIGHT TYPES OF VULNERABILITIES ON THE IR-ESCD DATASET

| Variants            | TP                     | BN                     | DE                     | EF                     | UC                     | RE                     | OF                     | SE                     |
|---------------------|------------------------|------------------------|------------------------|------------------------|------------------------|------------------------|------------------------|------------------------|
| ConSense(k-GNN)     | 93.16 <sup>+1.18</sup> | 94.30 <sup>+2.66</sup> | 91.18 <sup>+7.35</sup> | 88.41 <sup>+5.79</sup> | 99.42 <sup>-0.29</sup> | 99.73 <sup>+0.00</sup> | 97.45 <sup>+0.73</sup> | 99.38 <sup>+0.00</sup> |
| ConSense(GraphSAGE) | 92.45 <sup>+1.89</sup> | 94.30 <sup>+2.66</sup> | 91.18 <sup>+7.35</sup> | 88.41 <sup>+1.89</sup> | 99.42 <sup>-0.29</sup> | 99.73 <sup>+0.00</sup> | 97.09 <sup>+1.09</sup> | 99.38 <sup>+0.00</sup> |
| ConSense(GCN)       | 92.45 <sup>+1.89</sup> | 96.58 <sup>+0.38</sup> | 94.12 <sup>+4.41</sup> | 91.30 <sup>+2.90</sup> | 99.42 <sup>-0.29</sup> | 99.73 <sup>+0.00</sup> | 97.09 <sup>+1.09</sup> | 98.15 <sup>+1.23</sup> |
| ConSense(GIN)       | 92.22 <sup>+2.12</sup> | 96.20 <sup>+0.76</sup> | 92.65 <sup>+5.88</sup> | 91.30 <sup>+2.90</sup> | 99.42 <sup>-0.29</sup> | 99.73 <sup>+0.00</sup> | 97.09 <sup>+1.09</sup> | 99.38 <sup>+0.00</sup> |
| <b>ConSense</b>     | <b>94.34</b>           | <b>96.96</b>           | <b>98.53</b>           | <b>94.20</b>           | <b>99.13</b>           | <b>99.73</b>           | <b>98.18</b>           | <b>99.38</b>           |

TABLE VII  
PERFORMANCE COMPARISON (%) OF CONSENSE USING DIFFERENT GLOBAL INFORMATION EXCHANGE METHODS ON THE SCVHUNTER-SCD

| Methods           | Reentrancy   |              |              |              | Timestamp Dependency |              |              |              | Transaction State Dependency |              |              |              |
|-------------------|--------------|--------------|--------------|--------------|----------------------|--------------|--------------|--------------|------------------------------|--------------|--------------|--------------|
|                   | Acc          | Recall       | Precision    | F1           | Acc                  | Recall       | Precision    | F1           | Acc                          | Recall       | Precision    | F1           |
| ConSense(Average) | 81.33        | <b>92.50</b> | 82.13        | 86.87        | 76.95                | 81.67        | 79.09        | 79.97        | 87.43                        | 88.89        | 90.71        | 89.58        |
| ConSense(Sum)     | 80.33        | 92.00        | 81.43        | 86.23        | 76.90                | 83.33        | 79.90        | 80.08        | 87.09                        | 88.33        | 90.46        | 89.32        |
| ConSense(Partial) | 85.63        | 91.32        | 87.79        | 88.51        | 81.90                | 87.67        | 87.17        | 83.56        | 88.43                        | 91.00        | 90.21        | 90.81        |
| <b>ConSense</b>   | <b>88.33</b> | 90.69        | <b>92.31</b> | <b>91.14</b> | <b>85.71</b>         | <b>91.67</b> | <b>90.91</b> | <b>86.96</b> | <b>90.15</b>                 | <b>93.33</b> | <b>91.00</b> | <b>92.01</b> |

representation strategy in the NRE module, prioritizing its higher overall detection precision across most vulnerability types.

2) *Evaluating the Effectiveness of Local Feature Fusion Strategies*: Table VI presents the performance comparison of ConSense using different local feature fusion strategies in ExploreFormer across eight types of vulnerabilities. Traditional GNN-based methods, such as k-GNN, GraphSAGE, GCN, and GIN, exhibit notable drops in accuracy for vulnerabilities like DE and EF. This is because traditional GNNs rely solely on local information propagation, restricted by their receptive fields, which makes it difficult to capture information from distant nodes in complex smart contract graphs. For example, ConSense with k-GNN suffers a 7.35% drop in DE detection accuracy, and GraphSAGE shows a similar decline, indicating that these methods are limited in their ability to aggregate global context.

In contrast, ExploreFormer, integrated within the base ConSense framework, addresses this limitation by employing advanced attention mechanisms to capture both local and global relationships. ExploreFormer enhances node representations, making it more effective at detecting vulnerabilities by analyzing long-range dependencies in the contract's function call graph. This is evident in the higher detection rates of the base ConSense model, which achieves 98.53% accuracy for DE, significantly outperforming the variants. Furthermore, for vulnerabilities like unchecked external calls (UC), all variants show a minor improvement of 0.29%, but the overall accuracy across vulnerabilities such as DE, EF, and others consistently highlights the advantage of ExploreFormer. Its ability to combine local structure with global semantic information leads to more accurate detection, especially in cases where traditional GNNs struggle with complex contract structures. The result demonstrates that the ExploreFormer-based ConSense model is more adept at capturing both local and global context for improved vulnerability detection.

3) *Evaluating the Effectiveness of Global Information Exchange Mechanism*: Table VII demonstrates the impact of

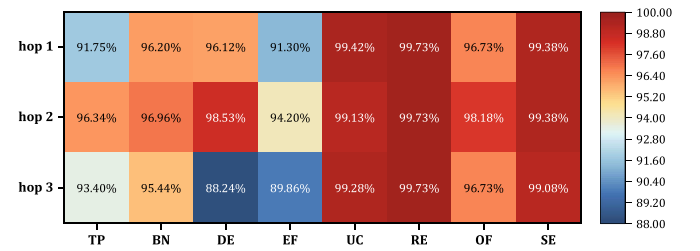


Fig. 6. Performance comparison (%) of ConSense by using different subgraph hops.

different global information exchange methods on the performance of ConSense, with a focus on the attention mechanism within ExploreFormer. The comparison includes various aggregation methods such as weighted average [47], weighted sum [51], and partial attention mechanism [52]. Experimental results show that employing the self-attention mechanism as the global information exchange method leads to a significant improvement in vulnerability detection performance. Specifically, for transaction state dependency detection, ExploreFormer with the global self-attention mechanism enhances accuracy and F1 score by 1.72% and 1.20%, respectively. Additionally, for reentrancy and timestamp dependency vulnerability detection, the accuracy improves by 2.70% and 3.81%, while the F1 scores increase by 2.63% and 3.40%, respectively. These improvements can be attributed to the ability of the self-attention mechanism to better capture both local and global structural information within the contract, which is crucial for detecting vulnerabilities closely tied to contract structure, such as reentrancy and timestamp dependency.

4) *Subgraph Hops*: Fig. 6 illustrates the performance of ConSense across eight types of smart contract vulnerabilities using different subgraph hop sizes in ExploreFormer. The subgraph hop size defines the range within which ExploreFormer extracts



node information, influencing both local and global context integration in the smart contract graph. A smaller hop size focuses on local information, while larger hop sizes aim to capture broader, global context from distant nodes.

The results show that the 2-hop setting achieves the best overall balance between accuracy and noise minimization. For example, with a 1-hop setting, the model struggles to gather sufficient global context, as seen in vulnerabilities like TP and DE, where the accuracy remains at 91.75% and 96.12%, respectively. These lower results indicate that a single hop may not be enough to capture complex relationships across the contract. However, with the 2-hop setting, both TP and DE see significant improvements, reaching 96.34% and 98.53% accuracy, respectively, demonstrating that additional contextual information from surrounding nodes leads to better detection without introducing too much noise. In contrast, the 3-hop setting introduces too much global information, which in some cases dilutes the relevance of the extracted features. For instance, EF and DE accuracy drop to 89.86% and 88.24%, respectively, from their higher values with 2-hop subgraphs. The decline suggests that beyond 2 hops, irrelevant or redundant information from distant nodes begins to negatively affect model performance, making it harder to distinguish critical patterns for these specific vulnerabilities. Thus, the 2-hop configuration provides an optimal balance, capturing enough global context for accurate detection while avoiding the performance drop seen with excessive information from larger subgraphs.

## V. DISCUSSION AND FUTURE WORKS

### A. Generalizability

ConSense's SC Graph Generator constructs the contract graph by treating functions as nodes and the calling relationships between them as edges, a design that avoids the need for vulnerability-specific expert knowledge. This enables ConSense to maintain strong generalizability across various types of vulnerabilities, as evidenced by the experimental results in Tables IV and V. The approach's ability to detect reentrancy, timestamp dependency, and transaction state dependency vulnerabilities, among others, highlights its adaptability to different vulnerability types without the need for extensive domain-specific customization.

However, one limitation of ConSense's generalizability stems from its reliance on the static analysis tool Slither to parse smart contracts and extract the necessary adjacency matrix information. While Slither provides valuable functionality for extracting calling relationships and contract structure, its effectiveness depends on the tool's support for various smart contract languages and blockchain platforms. As a result, the generalizability of ConSense is constrained by the range of languages and platforms compatible with Slither. Despite this, ConSense remains effective as long as the core contract code and function call relationships are accessible. To mitigate this limitation and further enhance ConSense's adaptability, future work will explore alternative parsing methods that do not rely on Slither, thereby broadening the tool's support for diverse blockchain ecosystems and smart contract languages.

### B. Threats to Validity

The validity of ConSense's results is influenced by both internal design choices and external data dependencies, each presenting distinct challenges. Internally, the framework's reliance on LLM-generated node features introduces a fundamental tension between semantic richness and computational practicality. While these features enable deep code understanding, they create preprocessing bottlenecks that may preclude real-time analysis without specialized hardware. This design constraint mirrors broader limitations in LLM-augmented security tools, where the trade-off between analytical depth and operational efficiency remains unresolved. Proposed distillation techniques offer a potential path forward, but their effectiveness in maintaining detection accuracy while reducing computational demands remains to be verified.

These internal limitations compound concerns about external validity stemming from ConSense's data requirements. The framework's data-driven approach, while powerful for well-represented vulnerability classes, struggles with rare or emerging threats due to inherent dataset imbalances. This is exemplified by its underperformance on integer overflow detection compared to oracle-based tools like IR-Fuzz, a direct consequence of sparse training samples for this vulnerability type. The recently introduced SCVHunter-SCD dataset helps mitigate this issue for evaluation purposes, but the fundamental challenge of generalizing to novel vulnerabilities persists. Future work on semi-supervised learning may relax the dependence on labeled data, but currently, this limitation positions ConSense as particularly effective for established vulnerability patterns rather than frontier threats.

## VI. CONCLUSION

This work introduces ConSense, a novel framework for detecting smart contract vulnerabilities that leverages both graph learning and large language models. ConSense comprises a dedicated graph generator, which constructs a comprehensive contract graph that preserves all elements and enriches nodes with semantic information, and ExploreFormer, a structure-aware graph learning module that captures long-range dependencies and integrates global context via advanced attention mechanisms. The conducting evaluations demonstrate ConSense's state-of-the-art performance across a range of vulnerability types, exhibiting both comprehensive accuracy and efficiency compared to existing approaches.

## REFERENCES

- [1] S. Wang, L. Ouyang, Y. Yuan, X. Ni, X. Han, and F.-Y. Wang, "Blockchain-enabled smart contracts: Architecture, applications, and future trends," *IEEE Trans. Syst., Man, Cybern. Syst.*, vol. 49, no. 11, pp. 2266–2277, Nov. 2019.
- [2] J. Huang, T. Huang, C. Dong, S. Duan, and Y. Pang, "Hierarchical network with local-global awareness for ethereum account de-anonymization," *IEEE Trans. Syst., Man, Cybern. Syst.*, to be published, doi: [10.1109/TSMC.2025.3571795](https://doi.org/10.1109/TSMC.2025.3571795).
- [3] C. F. Torres et al., "The art of the scam: Demystifying honeypots in ethereum smart contracts," in *Proc. 28th USENIX Secur. Symp.*, 2019, pp. 1591–1607.

- [4] H. Su, B. Guo, Y. Shen, and X. Suo, "Embedding smart contract in blockchain transactions to improve flexibility for the IoT," *IEEE Internet of Things J.*, vol. 9, no. 19, pp. 19073–19085, Oct. 2022.
- [5] W. Lian, Z. Bao, X. Zhang, B. Jia, and Y. Zhang, "A universal and efficient multi-modal smart contract vulnerability detection framework for big data," *IEEE Trans. Big Data*, vol. 11, no. 1, pp. 190–207, Feb. 2025.
- [6] Z. Li et al., "VulHunter: Hunting vulnerable smart contracts at EVM bytecode-level via multiple instance learning," *IEEE Trans. Softw. Eng.*, vol. 49, no. 11, pp. 4886–4916, Nov. 2023.
- [7] L. Luu, D.-H. Chu, H. Olickel, P. Saxena, and A. Hobor, "Making smart contracts smarter," in *Proc. 2016 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2016, pp. 254–269.
- [8] B. Mueller, "A framework for bug hunting on the ethereum blockchain," *ConsensSys/mythril*, 2017.
- [9] F. Ma et al., "Pluto: Exposing vulnerabilities in inter-contract scenarios," *IEEE Trans. Softw. Eng.*, vol. 48, no. 11, pp. 4380–4396, Nov. 2022.
- [10] P. Tsankov, A. Dan, D. Drachler-Cohen, A. Gervais, F. Buenzli, and M. Vechev, "Securify: Practical security analysis of smart contracts," in *Proc. 2018 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 67–82.
- [11] S. Kalra, S. Goel, M. Dhawan, and S. Sharma, "Zeus: Analyzing safety of smart contracts," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018, pp. 1–12.
- [12] A. Permenev, D. Dimitrov, P. Tsankov, D. Drachler-Cohen, and M. Vechev, "VerX: Safety verification of smart contracts," in *Proc. 2020 IEEE Symp. Secur. Privacy*, 2020, pp. 1661–1677.
- [13] S. Tikhomirov, E. Voskresenskaya, I. Ivanitskiy, R. Takhaviev, E. Marchenko, and Y. Alexandrov, "SmartCheck: Static analysis of ethereum smart contracts," in *Proc. 1st Int. Workshop Emerg. Trends Softw. Eng. Blockchain*, 2018, pp. 9–16.
- [14] J. Feist, G. Grieco, and A. Groce, "Slither: A static analysis framework for smart contracts," in *Proc. IEEE/ACM 2nd Int. Workshop Emerg. Trends Softw. Eng. Blockchain*, 2019, pp. 8–15.
- [15] Y. Xue, M. Ma, Y. Lin, Y. Sui, J. Ye, and T. Peng, "Cross-contract static analysis for detecting practical reentrancy vulnerabilities in smart contracts," in *Proc. 35th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2020, pp. 1029–1040.
- [16] B. Jiang, Y. Liu, and W. K. Chan, "ContractFuzzer: Fuzzing smart contracts for vulnerability detection," in *Proc. 33rd ACM/IEEE Int. Conf. Automated Softw. Eng.*, 2018, pp. 259–269.
- [17] Z. Liu et al., "Rethinking smart contract fuzzing: Fuzzing with invocation ordering and important branch revisiting," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 1237–1251, 2023.
- [18] T. D. Nguyen, L. H. Pham, J. Sun, Y. Lin, and Q. T. Minh, "sFuzz: An efficient adaptive fuzzer for solidity smart contracts," in *Proc. ACM/IEEE 42nd Int. Conf. Softw. Eng.*, 2020, pp. 778–788.
- [19] W. J.-W. Tann, X. J. Han, S. S. Gupta, and Y.-S. Ong, "Towards safer smart contracts: A sequence learning approach to detecting security threats," 2018, *arXiv: 1811.06632*.
- [20] S. Hochreiter, "Long short-term memory," in *Neural Computation*. Cambridge, MA, USA: MIT Press, 1997.
- [21] P. Qian, Z. Liu, Q. He, R. Zimmermann, and X. Wang, "Towards automated reentrancy detection for smart contracts based on sequential models," *IEEE Access*, vol. 8, pp. 19685–19695, 2020.
- [22] Y. Zhuang, Z. Liu, P. Qian, Q. Liu, X. Wang, and Q. He, "Smart contract vulnerability detection using graph neural network," in *Proc. Int. Joint Conf. Artif. Intell.*, 2020, pp. 3283–3290.
- [23] Z. Liu, P. Qian, X. Wang, Y. Zhuang, L. Qiu, and X. Wang, "Combining graph neural networks with expert knowledge for smart contract vulnerability detection," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 2, pp. 1296–1310, Feb. 2023.
- [24] P. Qian, Z. Liu, Y. Yin, and Q. He, "Cross-modality mutual learning for enhancing smart contract vulnerability detection on bytecode," in *Proc. ACM Web Conf.*, 2023, pp. 2220–2229.
- [25] F. Luo et al., "SCVHunter: Smart contract vulnerability detection based on heterogeneous graph attention network," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, 2024, pp. 1–13.
- [26] K. Zhang, Y. Zhu, J. Wang, and J. Zhang, "Adaptive structural fingerprints for graph attention networks," in *Proc. Int. Conf. Learn. Representations*, pp. 1–12, 2020.
- [27] C. F. Torres, J. Schütte, and R. State, "Osiris: Hunting for integer bugs in ethereum smart contracts," in *Proc. 34th Annu. Comput. Secur. Appl. Conf.*, 2018, pp. 664–676.
- [28] L. He, X. Zhao, and Y. Wang, "ParSE: Efficient detection of smart contract vulnerabilities via parallel and simplified symbolic execution," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, 2024, pp. 272–273.
- [29] Y. Xue et al., "xFuzz: Machine learning guided cross-contract fuzzing," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 2, pp. 515–529, Mar./Apr. 2024.
- [30] J. He, M. Balunović, N. Ambroladze, P. Tsankov, and M. Vechev, "Learning to fuzz from symbolic execution with application to smart contracts," in *Proc. 2019 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 531–548.
- [31] Y. Pang et al., "Slim UNETR: Scale hybrid transformers to efficient 3D medical image segmentation under limited computational resources," *IEEE Trans. Med. Imag.*, vol. 43, no. 3, pp. 994–1005, Mar. 2024.
- [32] T. Huang et al., "AdaptFormer: An adaptive hierarchical semantic approach for change detection on remote sensing images," *IEEE Trans. Instrum. Meas.*, vol. 73, 2024, Art. no. 5502612.
- [33] L. Li, Y. Liu, G. Sun, and N. Li, "Smart contract vulnerability detection based on automated feature extraction and feature interaction," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 9, pp. 4916–4929, Sep. 2024.
- [34] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," 2014, *arXiv:1412.3555*.
- [35] W. Wang, J. Song, G. Xu, Y. Li, H. Wang, and C. Su, "ContractWard: Automated vulnerability detection models for ethereum smart contracts," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 1133–1144, Second Quarter, 2021.
- [36] C. Sendner et al., "Smarter contracts: Detecting vulnerabilities in smart contracts with deep transfer learning," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, pp. 1–18, 2023.
- [37] Y. Sun et al., "GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, 2024, pp. 2048–2060.
- [38] H. Ding, Y. Liu, X. Piao, H. Song, and Z. Ji, "SmartGuard: An LLM-enhanced framework for smart contract vulnerability detection," *Expert Syst. Appl.*, vol. 269, 2025, Art. no. 126479.
- [39] Y. Zhuang, Z. Liu, P. Qian, Q. Liu, X. Wang, and Q. He, "Smart contract vulnerability detection using graph neural networks," in *Proc. 29th Int. Conf. Int. Joint Conf. Artif. Intell.*, 2021, pp. 3283–3290.
- [40] Y. Wang, X. Zhao, L. He, Z. Zhen, and H. Chen, "ContractGNN: Ethereum smart contract vulnerability detection based on vulnerability sub-graphs and graph neural networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 6, pp. 6382–6395, Nov./Dec. 2024.
- [41] Y. Zhang et al., "An efficient smart contract vulnerability detector based on semantic contract graphs using approximate graph matching," *IEEE Internet of Things J.*, vol. 10, no. 24, pp. 21431–21442, Dec. 2023.
- [42] Y. Pang et al., "Sparse-Dyn: Sparse dynamic graph multirepresentation learning via event-based sparse temporal attention network," *Int. J. Intell. Syst.*, vol. 37, no. 11, pp. 8770–8789, Jul. 2022.
- [43] J. Cai, B. Li, J. Zhang, X. Sun, and B. Chen, "Combine sliced joint graph with graph neural networks for smart contract vulnerability detection," *J. Syst. Softw.*, vol. 195, 2023, Art. no. 111550.
- [44] B. Roziere et al., "Code Llama: Open foundation models for code," 2023, *arXiv:2308.12950*.
- [45] T. Durieux, J. F. Ferreira, R. Abreu, and P. Cruz, "Empirical review of automated analysis tools on 47,587 ethereum smart contracts," in *Proc. ACM/IEEE 42nd Int. Conf. Softw. Eng.*, 2020, pp. 530–541.
- [46] Y. Pang et al., "Graph decipher: A transparent dual-attention graph neural network to understand the message-passing mechanism for the node classification," *Int. J. Intell. Syst.*, vol. 37, no. 11, pp. 8747–8769, Jul. 2022.
- [47] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.
- [48] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 8024–8035.
- [49] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch geometric," 2019, *arXiv:1903.02428*.
- [50] I. Loshchilov et al., "Fixing weight decay regularization in adam," 2017, *arXiv:1711.05101*.
- [51] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?," 2018, *arXiv:1810.00826*.
- [52] X.-T. Vo, D.-L. Nguyen, A. Priadana, and K.-H. Jo, "Efficient vision transformers with partial attention," in *Proc. Eur. Conf. Comput. Vis.*, Springer, 2024, pp. 298–317.